

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

# Understanding Task Subtypes

A subset of the tasks should be aligned to subtypes that target key challenge areas. Tasks can be aligned with multiple subtypes if they span multiple challenge areas.

If your task aligns with none of these subcategories, then leave the **subcategories** field in your task.toml blank.

These subcategories are defined as:

## 1. Long Context

Tasks that require models to test their context windows by reading large documents. File must be at least **50k tokens** and cannot simply be parsed programmatically or through keyword search by the agent.

## Supported Formats & Examples

- **Text-Heavy Formats:**
  - **PDF:** Academic papers (50–150 pages), regulatory filings (10-K), or ISO technical standards.
  - **DOCX:** Legal contracts with deep appendices or internal architecture proposals.
  - **Markdown/TXT:** Multi-day meeting transcripts or massive technical handbooks.
- **Semi-Structured Content:**
  - **HTML:** Entire documentation sites scraped into a single reference file.
  - **JSON/YAML:** Massive API schemas or nested configuration files with inline documentation.
  - **CSV:** Wide datasets with extensive metadata headers and notes.

- **Conversational/Narrative:**

- **Chat Logs:** Slack/Discord exports or weeks of customer support transcripts.
- **Email:** Long back-and-forth threads where the agent must track referenced attachments.

**Threshold:** Files must be at least **50k tokens** and cannot simply be parsed programmatically or through keyword search. The task must rely on the model's semantic understanding of the provided document.

[Long Context Task Checklist](#) — Use this before submitting or reviewing any `long_context` task.

## 2. Tool Specific

Tasks that target tools that provide SDK & APIs where models generally underperform.

- **Examples:** Blender, FFmpeg, ImageMagick, Graphviz, MLFlow, WandB, Prefect, Superset, GIMP, QGIS, etc.
- **Goal:** Providing exercises for these tools is highly useful for identifying agent blindspots in specialized workflows.

## 3. API Integration

Tasks that involve building, interacting with, or debugging APIs to solve a task. These are specifically tasks where the API source code is included in the environment.

- **Implementation:** APIs must be mocked within the Docker environment without external dependencies.
- **Interaction:** The agent interacts strictly via the terminal (no MCP). Source code for the API must be included in the environment.
- **Frameworks:** Flask, Ruby on rails, Rustapi, spring boot, django, express js, fastify, play, gin and more.

- Avoid using FastAPI too much. This is used heavily throughout other datasets and risk oversaturation of the use of this framework if not avoided.

## 4. DB Interaction

Tasks that involve gathering context and/or problem solving through interacting with a database. This will avoid tasks where the agent can read the underlying data without directly interacting with the DB.

- **DB Types:** SQL, NoSQL, Vector Databases, In Memory Databases, and more.
- **The "Flat-File" Limit:** DBs represented purely as CSVs should make up minority of DB-based tasks to ensure agents are actually interacting with database engines.

## 4. User Interface Building

Tasks that create, edit, or update a user interface.

- **Verification:** UI tasks must still use Python pytest validators. If browser automation is needed, use Playwright's Python bindings from pytest rather than a JavaScript or TypeScript Playwright suite.

**Ready to build?** [View the Task Authoring Templates](#)

---

PREVIOUS

[NEW: Milestones](#)

NEXT

[NEW: Rubrics](#)